

OrderFlow: CDC-Driven E-Commerce ELT Pipeline

Product Requirements Document (PRD) — Agent-Buildable Spec

Data Engineering Portfolio Project

September 2026

1. Overview

1.1 Purpose of this document

This PRD is written to be handed directly to an AI coding agent (Claude Code, Antigravity, Cursor, etc.) so it can scaffold, implement, and verify the **OrderFlow** project with minimal ambiguity. It defines scope, architecture, data model, folder structure, build phases with explicit acceptance criteria, and a definition of done.

1.2 Business objective

OrderFlow simulates a real e-commerce platform’s operational database (customers, products, orders, inventory) and builds a **Change Data Capture (CDC)-driven ELT pipeline** that captures only the *changes* happening in that database — not full re-extraction — and flows them into an analytics warehouse. The warehouse models key dimensions using **Slowly Changing Dimension Type 2 (SCD2)**, preserving full history of how customer and product data changed over time (e.g., every price change, every address update), rather than only showing the current state. **Apache Airflow** orchestrates the pipeline, and **Great Expectations** validates data quality at the source before it enters the warehouse, complementing dbt’s business-logic tests downstream.

1.3 Why this project exists (relative to a prior streaming project)

This project intentionally uses a different toolset than a prior real-time streaming/Spark lakehouse project, to demonstrate breadth rather than repetition:

Prior project pattern	OrderFlow pattern
Event-stream simulation (Kafka producer)	Database-level Change Data Capture
AWS Step Functions orchestration	Apache Airflow orchestration
	dbt tests + Great Expectations

dbt tests only	(source-level validation)
Fact/aggregate dbt models	SCD Type 2 dimensional modeling (history tracking)
Athena/Glue-style warehouse	DuckDB locally, portable to Snowflake/BigQuery

1.4 Success criteria

- PostgreSQL runs as a realistic OLTP source with ongoing simulated activity (inserts, updates, deletes).
- Change events are captured with before/after state and operation type (insert/update/delete), either via Debezium or a schema-faithful fallback simulator.
- Airflow DAGs orchestrate ingestion → warehouse load → dbt run, with retries configured and failure alerting demonstrably working (not just claimed).
- `dim_customers_scd2` and `dim_products_scd2` correctly implement SCD Type 2: an update to a tracked attribute closes out the old row (`valid_to` set, `is_current` = false) and inserts a new current row — proven with an explicit test, not just asserted in a report.
- Great Expectations validates source data before warehouse load; dbt tests validate business logic after transformation.
- The entire pipeline runs locally with zero cloud credentials required.
- A validation script independently proves SCD2 correctness, idempotency, and CDC completeness — not just a pipeline run log claiming success.

1.5 Non-goals (out of scope for v1)

- Real production Snowflake/BigQuery deployment (local DuckDB only for v1; cloud config documented for later).
- Real payment processing, authentication, or a user-facing application.
- Handling of true multi-terabyte scale or high-availability Kafka clustering.
- ML-based demand forecasting or recommendation systems.

2. Architecture

2.1 High-level data flow

```

[PostgreSQL OLTP DB]
  customers, products, orders, order_items, inventory
    |
    v (simulate_changes.py generates ongoing
INSERT/UPDATE/DELETE activity)
    |
[ CDC Capture Layer ]
  Debezium + Kafka Connect (reads Postgres WAL)
  OR fallback: Python CDC simulator emitting Debezium-shaped JSON
    |
    v (before/after state + operation type: c/u/d)

```

```

[Raw Change Log] (local JSON/Parquet, partitioned by table + date)
  |
  v (Airflow DAG: cdc_ingestion_dag)
[Airflow: warehouse_load_dag]
  Applies changes to DuckDB warehouse tables (upsert/merge logic)
  |
  v
[DuckDB Warehouse – Staging Layer]
  Cleaned, typed views per source table
  |
  v (Airflow DAG: dbt_run_dag)
[dbt Transformation Layer]
  dim_customers_scd2, dim_products_scd2 (SCD Type 2 history)
  fct_orders (order fact table)
  agg_daily_sales (business aggregates)
  |
  v
[Great Expectations] <-- validates source data BEFORE warehouse
load
[dbt tests] <-- validates business logic AFTER
transformation

```

2.2 Component roles

Component	Role
PostgreSQL	Simulated OLTP source of truth; realistic schema with ongoing changes
Debezium / CDC simulator	Captures row-level changes with before/after state, not full snapshots
Apache Airflow	Orchestrates ingestion, warehouse load, and dbt runs; handles retries, SLAs, alerting
DuckDB	Local embedded analytics warehouse (portable to Snowflake/BigQuery later)
dbt (dbt-duckdb)	Transformation layer; implements SCD Type 2 dimensional models
Great Expectations	Source-level data quality validation before data enters the warehouse

3. Data model

3.1 Source tables (PostgreSQL)

customers: customer_id (PK), name, email, address, created_at, updated_at

products: product_id (PK), name, category, price, updated_at

orders: order_id (PK), customer_id (FK), order_date, status, total_amount - status lifecycle: pending -> shipped -> delivered or pending -> cancelled

order_items: order_item_id (PK), order_id (FK), product_id (FK), quantity, unit_price

inventory: product_id (FK), warehouse_id, stock_level, updated_at

3.2 Simulated change activity

postgres/simulate_changes.py must continuously generate realistic activity: - New orders created at a configurable rate - Order status transitions over time (not instant — simulate realistic delay between pending→shipped→delivered) - Product price changes (periodic, simulating promotions/repricing) - Customer address/email updates (occasional) - Inventory stock level fluctuations (decrement on order, periodic restock)

3.3 CDC event shape

Each captured change event must include, at minimum:

```
{
  "table": "products",
  "operation": "update",
  "before": { "product_id": 101, "price": 29.99, "updated_at": "..."},
  "after": { "product_id": 101, "price": 24.99, "updated_at": "..."},
  "captured_at": "ISO-8601 timestamp"
}
```

operation must be one of insert, update, delete. before is null for inserts; after is null for deletes.

3.4 Warehouse models (dbt)

dim_customers_scd2 — one or more rows per customer, one per distinct historical state: - Columns: customer_key (surrogate), customer_id, name, email, address, valid_from, valid_to, is_current - When a tracked attribute (address, email) changes: close the current row (valid_to = change timestamp, is_current = false), insert a new row (valid_from = change timestamp, valid_to = NULL, is_current = true)

dim_products_scd2 — same SCD2 pattern, tracking price and category changes over time.

fct_orders — one row per order, joined to the *current* dimension state at time of order (or the state valid at order_date, if implementing point-in-time correctness — document which approach is used and why).

agg_daily_sales — daily revenue, order count, average order value, cancellation rate.

3.5 Data quality split

- **Great Expectations** (before warehouse load): validate raw/staged source data — e.g., no null primary keys, price > 0, valid status enum values, referential keys present.
- **dbt tests** (after transformation): validate business logic — e.g., exactly one is_current = true row per customer/product, valid_to > valid_from, no overlapping validity windows, fact table foreign keys resolve to a dimension row.

4. Repository structure

```
orderflow-cdc-pipeline/
├── README.md
├── PRD.md
├── docker-compose.yml
├── postgres/
│   ├── init.sql
│   └── simulate_changes.py
├── cdc/
│   ├── debezium_connector_config.json
│   └── cdc_event_schema.py
├── airflow/
│   ├── dags/
│   │   ├── cdc_ingestion_dag.py
│   │   ├── warehouse_load_dag.py
│   │   └── dbt_run_dag.py
│   └── plugins/
├── great_expectations/
│   └── expectations/
├── dbt_project/
│   ├── dbt_project.yml
│   ├── models/
│   │   ├── staging/
│   │   └── marts/
│   │       ├── dim_customers_scd2.sql
│   │       ├── dim_products_scd2.sql
│   │       ├── fct_orders.sql
│   │       └── agg_daily_sales.sql
│   └── schema.yml
├── scripts/
│   ├── run_full_pipeline.py
│   └── validate_scd2.py
└── docs/
    └── architecture.md
```

5. Build phases (with explicit acceptance criteria)

Each phase must be independently verified — with real command output, not a narrated summary — before moving to the next.

Phase 0 — Repo & foundations

- Scaffold folder structure above; README.md with setup instructions.
- **Acceptance:** repo structure matches §4; docker-compose.yml present and valid.

Phase 1 — PostgreSQL source + change simulator

- init.sql creates schema and seed data (at least 50 customers, 30 products, 100 orders).
- simulate_changes.py runs continuously, producing realistic inserts/updates/deletes at a configurable rate.
- **Acceptance:** running the simulator for 60 seconds produces verifiable changes in Postgres (query row counts and updated_at timestamps before/after).

Phase 2 — CDC capture

- Implement Debezium + Kafka Connect, OR a Python fallback simulator emitting Debezium-shaped JSON per §3.3.
- **Explicitly state which approach was used and why** (e.g., “Debezium was too resource-heavy for local Docker; used a WAL-polling Python fallback that produces identical event shape”).
- **Acceptance:** captured change-log file/table shows real before/after state matching what was actually changed in Postgres — verify at least 3 specific changes manually.

Phase 3 — Airflow orchestration

- Three DAGs per §4: ingestion, warehouse load, dbt run.
- Configure retries (e.g., 2 retries with exponential backoff) and at least one alerting mechanism (email or logged Slack-webhook stub).
- **Acceptance:** deliberately break one task (e.g., point it at a bad file path), confirm the DAG retries the configured number of times and then fails visibly with an alert — show the actual Airflow UI/log output, not just a description.

Phase 4 — Warehouse load & SCD2 modeling

- warehouse_load_dag applies captured changes to DuckDB staging tables (upsert/merge, not blind append).
- dbt builds dim_customers_scd2 and dim_products_scd2.
- **Acceptance (critical):** run scripts/validate_scd2.py which must prove:
 1. Exactly one is_current = true row exists per customer_id/product_id.
 2. No overlapping [valid_from, valid_to) windows for the same entity.
 3. When a source price/address change is simulated, a NEW row appears and the OLD row is closed (not overwritten) — verified by comparing row counts before and after a specific, deliberate change.

Phase 5 — Data quality (Great Expectations + dbt tests)

- Great Expectations suite validates source-level data before warehouse load (nulls, ranges, enums, referential presence).
- dbt schema tests validate business logic (uniqueness, relationships, `assert_one_current_row_per_entity`, `assert_no_overlapping_scd2_windows`).
- **Acceptance:** intentionally inject bad source data (e.g., negative price, null `customer_id`) and confirm Great Expectations catches it before it reaches the warehouse — show the actual validation failure output.

Phase 6 — End-to-end orchestrator & idempotency proof

- `scripts/run_full_pipeline.py` runs the entire chain and logs stage-by-stage status.
- **Acceptance:** run the full pipeline twice in a row without regenerating source changes. Row counts in fact/dimension tables must NOT double — prove idempotency explicitly, the same way RideStream's dedup was verified.

6. Non-functional requirements

Category	Requirement
Reproducibility	Entire stack runs via docker-compose up + one orchestrator script, no cloud credentials
Idempotency	Re-running any stage on the same data must not duplicate rows
Observability	Airflow UI shows DAG run history, retries, and failures; structured logging throughout
Data quality	Two-layer validation: Great Expectations (source) + dbt tests (business logic)
Code quality	Type-hinted Python, docstrings, unit tests for CDC parsing and SCD2 merge logic
Portability	DuckDB warehouse layer swappable for Snowflake/BigQuery without changing dbt model logic (only profile/connection changes)

7. Verification checklist (definition of done)

- ☐ Repo scaffolded per §4
- ☐ Postgres seeded and continuously generating realistic changes
- ☐ CDC events captured with accurate before/after state (approach explicitly documented)
- ☐ Airflow DAGs run ingestion → warehouse load → dbt, with proven retry/alert behavior
- ☐ dim_customers_scd2 / dim_products_scd2 pass all three SCD2 correctness checks in §5 Phase 4
- ☐ Great Expectations catches injected bad data before warehouse load (demonstrated, not claimed)
- ☐ dbt tests pass, including custom SCD2 integrity tests
- ☐ Full pipeline is idempotent — proven by running it twice and comparing row counts
- ☐ README documents the Debezium-vs-fallback decision and how to run everything locally
- ☐ docs/architecture.md explains how this would map to Snowflake/BigQuery + managed Airflow (MWAA/Composer) in production

8. Open questions for the user (agent should ask before assuming)

1. Debezium/Kafka Connect (heavier, more “real”) or a Python CDC simulator (lighter, faster to build) — any preference, or should the agent decide based on what runs reliably in this environment?
2. Should fct_orders join to the *current* dimension state or the *point-in-time* dimension state valid at order_date? (Point-in-time is more advanced and more resume-worthy, but more complex to implement correctly.)
3. Target simulated data volume — is ~100-200 orders enough for a demo, or should the simulator run longer to produce a larger dataset?
4. Any interest in eventually pointing this at a real free-tier Snowflake or BigQuery sandbox, or should this stay DuckDB-only?